

PHASE 2 — APIs & TOOLS

Week 3

AI APIs & LangChain Intro

Calling AI from code like a professional

API auth & key
management

Request / response
structure

Streaming responses

Multi-turn conversation
history

LangChain chains &
pipes

Memory chatbot

Week 3 at a Glance

Two sessions • Raw API → LangChain → Memory chatbot

Session 1

Working with AI APIs

- API authentication & key management
- Request/response structure & the SDK
- Streaming responses (tail -f analogy)
- Multi-turn: manual history management

Session 2

LangChain & Memory Chatbot

- LangChain chains — the pipe operator
- Prompt templates with variables
- RunnableWithMessageHistory
- Lab: build your domain chatbot



Multi-turn chatbot with conversation memory — customised for your real work domain — 6+ turns

The AI API Is Just a REST API

Everything you know about API integration already applies here.

Any REST API you've used

Method:	POST
Endpoint:	<code>https://api.service.com/v1/resource</code>
Auth:	<code>Authorization: Bearer sk-...</code>
Body (JSON):	<code>{ model, payload, options }</code>
Response:	<code>{ result, usage, metadata }</code>

Ollama API

Method:	POST
Endpoint:	<code>http://localhost:11434/api/chat</code>
Auth:	<code>x-api-key: sk-ant-...</code>
Body (JSON):	<code>{ model, messages[], max_tokens }</code>
Response:	<code>{ content[], usage, stop_reason }</code>

💡 IT Analogy

An AI API call is no different from calling a payment gateway, a weather API, or a Jira REST endpoint.

Authenticate with a key → POST JSON → parse the JSON response. The SDK wraps the HTTP so you don't write curl commands.

Streaming Responses

Receive tokens as they are generated — don't wait for the full response.



Standard call

Waits for the full response before returning.
User sees nothing until generation is complete.
Safe for short responses and batch processing.

```
response = client.messages.create(  
    model=MODEL,  
    max_tokens=300,  
    messages=[...]  
)  
  
text = response.content[0].text
```



Streaming call

Returns tokens as they arrive.
User sees text appearing in real time.
Use for long responses and interactive UIs.

```
with client.messages.stream(  
    model=MODEL,  
    max_tokens=300,  
    messages=[...]  
) as stream:  
    for text in stream.text_stream:  
        print(text, end="", flush=True)
```

Multi-Turn Conversations

The API is always stateless. YOU maintain the state.

Turn 1

messages list: 2 messages



We run Windows Server 2019.



Great. Patch cadence recommendation: monthly...

↑ *user message appended* ↑ *assistant reply appended*

↓ *full history sent again*

Turn 2

messages list: 4 messages



What OS did I mention?



You mentioned Windows Server 2019.

↑ *user message appended* ↑ *assistant reply appended*

↓ *full history sent again*

Turn 3

messages list: 6 messages



Risk if we skip 3 months?



Critical: ransomware exposure, compliance...

↑ *user message appended* ↑ *assistant reply appended*

LangChain — The Pipe Operator

Compose AI steps like Unix pipes. Swap models without rewriting your chain.

```
Unix:      cat access.log | grep ERROR | wc -l
LangChain: prompt         | llm         | StrOutputParser()
```



ChatPromptTemplate

Reusable prompt
with {variables}.
Like a parameterised
SQL query.



ChatOllama (LLM)

The AI model.
Swap this one line
to change provider
(OpenAI, Ollama...).



StrOutputParser

Extracts plain text
from AIMessage.
Use JsonOutputParser
for JSON output.

```
llm = ChatOllama(model=MODEL, temperature=0.2, max_tokens=512)

chain = prompt | llm | StrOutputParser()           # pipe operator

result = chain.invoke({"domain": "...", "question": "..."}) # run the chain
```

Memory Chatbot Architecture

RunnableWithMessageHistory = session middleware. You call .invoke() — it handles state.



User input + session_id

session_id = 'user-1' (like a cookie/session token)



Load session history

get_session_history(session_id) → InMemoryChatMessageHistory



Inject into prompt via MessagesPlaceholder

Past messages fill the {history} slot in ChatPromptTemplate



Call LLM (ChatOllama)

Full prompt with history sent to Ollama



Save response back to session store

Both user message and AI reply added to history automatically

Lab: Your Domain Chatbot

Customise the memory chatbot for your real work domain. 6+ conversation turns.

1

Set MY_DOMAIN

Change to your actual area: Azure cloud, SQL Server DBA, network security, DevOps...

2

Customise system prompt

Add 2–3 rules: 'Flag security risks', 'Include cost estimates', 'Assume team of 5 engineers'

3

Write 6+ real turns

Use actual questions you face at work — not generic examples. The chatbot should remember turn 1 when answering turn 4+

4

Verify memory works

Turn 3: ask 'what did I mention earlier?' — the bot should correctly recall info from turns 1 and 2

**Rubric:**

Notebook runs top-to-bottom · MY_DOMAIN changed · 6+ turns written · Memory verified in turn 3+



Stretch 1: Add `clear_history()` function



Stretch 2: Run 2 sessions for different roles — compare answers

After Week 3 you will be able to:

Call AI from code like a professional.

- ✦ Authenticate and call the Ollama API — request, stream, parse
- ✦ Build multi-turn conversations with manual history management
- ✦ Use LangChain to compose prompt → LLM → parser in one pipe
- ✦ Wrap any chain with RunnableWithMessageHistory for session memory
- ✦ Run isolated chatbot sessions for different users and domains



Homework

1. Change MY_DOMAIN to your real work area
2. Write 6+ real conversation turns from your job
3. Stretch: add clear_history() + a second session
4. Install before Week 4: pip install chromadb sentence-transformers